# AETHER

Autonomous Multi-Agent Urban Simulation &
Decision Intelligence Platform

SOFTWARE REQUIREMENTS SPECIFICATION
Cahier des Charges

Project Creators

Louay Hamdi & Yassine Mezghani

Technical Domain
Java · Artificial Intelligence · Big Data · Multi-Agent Simulation · Data Visualization

Version 1.0
Academic Project Specification — 2026

“Simulate. Understand. Predict. Transform.”

# Table of Contents

1. Foreword
2. Executive Summary
3. General Introduction
3.1 Presentation
3.2 Motivation
4. Context & Problem Statement
4.1 Central Problem
4.2 Sub-Problems
5. Project Objectives
5.1 General Objective
5.2 Specific Objectives
6. Requirements Analysis
6.1 Stakeholders & Actors
6.2 Functional Requirements
6.3 Non-Functional Requirements
7. Technical Architecture
7.1 Architectural Overview
7.2 Logical Layers
8. Module Descriptions
9. Technologies & Tools
10. Constraints & System Specifications
11. Project Planning
12. Testing & Validation Strategy
13. Deliverables
14. Risk Management
15. Conclusion
16. Bibliography & References
Appendix A — Glossary
Appendix B — Conceptual UML

# 1. Foreword

This document represents the formal Software Requirements Specification (SRS) for project AETHER, developed by Louay Hamdi and Yassine Mezghani. The project focuses on advanced Java development, multi-agent simulation, artificial intelligence, data processing, and interactive visualization.

The specification defines the objectives, requirements, architecture, technologies, constraints, validation strategy, and expected deliverables of the system.

# 2. Executive Summary

AETHER is an interactive urban simulation platform that creates a dynamic virtual city populated by autonomous citizens, vehicles, businesses, and infrastructure.

The system continuously simulates interactions between these entities and records the resulting events. Users can observe the city in real time, accelerate the simulation, inspect entities, introduce major events, and compare alternative futures using a What-If simulation engine.

The central idea is to create a computational laboratory where complex urban behavior can emerge from simple agent-level rules rather than being manually scripted.

# 3. General Introduction

## 3.1 Presentation

AETHER (Autonomous Multi-Agent Urban Simulation & Decision Intelligence Platform) provides a living virtual representation of an imaginary city. It combines a Java simulation engine, autonomous agents, graph-based movement, economic and transportation models, persistent event data, and interactive visualization.

## 3.2 Motivation

Cities are complex systems in which transportation, population, businesses, energy consumption, and infrastructure influence one another. AETHER provides a safe environment for studying these interactions without affecting a real city.

The project is motivated by the need to demonstrate how software engineering, algorithms, artificial intelligence, and large-scale data processing can be combined in a single practical Java application.

# 4. Context & Problem Statement

## 4.1 Central Problem

How can a Java-based system model a continuously evolving city in which thousands of autonomous entities interact, while allowing users to observe, analyze, and compare the consequences of different decisions?

## 4.2 Sub-Problems

• Efficiently simulating thousands of autonomous agents without excessive performance degradation.

• Modeling realistic interactions between citizens, vehicles, businesses, infrastructure, and the environment.

• Managing large volumes of simulation events and historical data.

• Providing graph-based navigation and dynamic traffic behavior.

• Creating reproducible alternative scenarios from the same initial state.

• Visualizing a complex simulation in real time without overwhelming the user interface.

• Using AI and statistical analysis to identify trends and make predictions from simulation data.

# 5. Project Objectives

### 5.1 General Objective

To design and implement a Java-based multi-agent urban simulation platform capable of modeling a dynamic virtual city, processing large volumes of events, visualizing its evolution in real time, and evaluating alternative scenarios.

### 5.2 Specific Objectives

• Implement autonomous citizen, vehicle, and business agents.

• Create a configurable virtual city containing roads, neighborhoods, buildings, services, and economic zones.

• Implement graph-based navigation and pathfinding using algorithms such as A*.

• Develop a concurrent simulation engine capable of running thousands of agents.

• Generate and process a continuous stream of simulation events.

• Store important simulation states and historical events.

• Implement a What-If engine based on state snapshots and scenario branching.

• Provide real-time dashboards and an interactive city visualization.

• Implement statistical or machine-learning models for selected predictions.

• Provide replay and comparison tools for analyzing simulation outcomes.

## 6. Requirements Analysis

### 6.1 Stakeholders & Actors

Administrator: Configures the city, simulation parameters, population size, and simulation speed.

Analyst: Runs scenarios, compares indicators, analyzes historical data, and studies simulation results.

Observer: Explores the city, views live events, and inspects entities without changing core configuration.

### 6.2 Functional Requirements

#### FR1 — City Configuration

The system shall allow users to create or load a virtual city containing neighborhoods, roads, buildings, services, and economic zones.

#### FR2 — Multi-Agent Simulation

The system shall simulate autonomous citizens, vehicles, and businesses whose behavior depends on their internal state and environment.

#### FR3 — Citizen Behavior

Citizens shall be able to perform activities such as commuting, working, shopping, consuming resources, changing jobs, and moving between locations.

#### FR4 — Vehicle & Traffic Simulation

Vehicles shall navigate the road network, select routes, respond to congestion, and contribute to traffic conditions.

#### FR5 — Business & Economy Simulation

Businesses shall manage employees, revenue, expenses, demand, and operational status, with changes influencing the simulated economy.

#### FR6 — Event Processing

The system shall generate, process, and store important simulation events such as movements, transactions, job changes, and infrastructure events.

### FR7 — Real-Time Visualization

The system shall display the evolving city, major entities, indicators, and live events while the simulation is running.

### FR8 — Simulation Controls

Users shall be able to start, pause, resume, reset, and accelerate the simulation using configurable speed levels.

### FR9 — What-If Scenarios

Users shall be able to create alternative scenarios from a selected simulation state and introduce changes such as road construction, factory closure, fuel-price changes, or public-transport improvements.

### FR10 — Scenario Comparison

The system shall compare baseline and alternative simulations using indicators such as traffic, employment, population, energy consumption, and economic activity.

### FR11 — Historical Replay

The system shall allow users to inspect and replay recorded simulation events and selected historical states.

### FR12 — Analytics & Prediction

The system shall provide statistical analysis and, where implemented, machine-learning predictions based on simulation data.

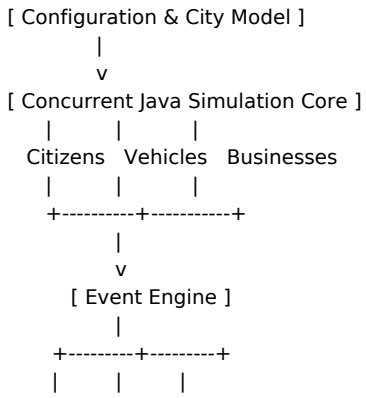
## 6.3 Non-Functional Requirements

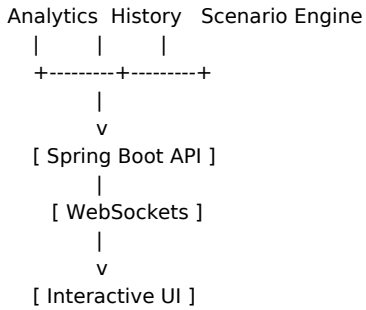
| Code | Category | Requirement |
|---|---|---|
| NFR-01 | Performance | Support a target configuration of at least 10,000 citizens, 5,000 vehicles, and 1,000 businesses o |
| NFR-02 | Concurrency | The simulation shall use thread-safe mechanisms and avoid deadlocks and race conditions. |
| NFR-03 | Responsiveness | Normal dashboard interactions should remain responsive while the simulation is running. |
| NFR-04 | Scalability | The architecture shall permit increasing agent and event counts without redesigning the entire ap |
| NFR-05 | Reliability | A failure in an individual agent should not terminate the complete simulation. |
| NFR-06 | Reproducibility | The simulation shall support deterministic runs when the same configuration and random seed ar |
| NFR-07 | Maintainability | The system shall follow modular design, SOLID principles, and clear separation of responsibilities. |
| NFR-08 | Usability | The main simulation controls, indicators, and scenario tools shall be accessible from a unified inte |

# 7. Technical Architecture

## 7.1 Architectural Overview

AETHER follows a modular, event-driven architecture centered on a concurrent Java simulation core.






### 7.2 Logical Layers

Domain & Simulation Layer: Agents, city entities, rules, simulation clock, and state transitions.

Algorithm Layer: Graph representation, A* pathfinding, traffic calculations, and optimization logic.

Event & Data Layer: Event generation, persistence, historical records, and analytics.

Decision Intelligence Layer: Statistical analysis, prediction models, and What-If scenario evaluation.

Application Layer: Spring Boot REST services and WebSocket communication.

Presentation Layer: Interactive dashboard and city visualization.

## 8. Module Descriptions

| Module | Main Responsibility |
|---|---|
| MOD-01 CityManager | Creates and manages city topology, neighborhoods, buildings, roads, and services. |
| MOD-02 AgentEngine | Creates and updates autonomous citizen, vehicle, and business agents. |
| MOD-03 RoutingEngine | Maintains the road graph and calculates routes using A* or another suitable pathfinding algorithm. |
| MOD-04 TrafficEngine | Calculates road usage, congestion, travel time, and transportation effects. |
| MOD-05 EconomyEngine | Models business activity, employment, income, demand, and economic indicators. |
| MOD-06 EventEngine | Generates, distributes, filters, and stores simulation events. |
| MOD-07 WhatIfEngine | Creates state snapshots, applies scenario changes, executes branches, and compares results. |
| MOD-08 AnalyticsEngine | Calculates KPIs, trends, statistics, and optional predictive models. |
| MOD-09 ReplayEngine | Stores and replays important historical events and simulation states. |
| MOD-10 CockpitUI | Displays the city, live events, indicators, entity details, and simulation controls. |

## 9. Technologies & Tools

| Area | Proposed Technology |
|---|---|
| Core language | Java 21 LTS |
| Backend | Spring Boot 3.x |
| Build & dependencies | Apache Maven |
| Database | PostgreSQL |
| Real-time communication | WebSockets |
| Frontend | React + TypeScript, or JavaFX if a desktop-only implementation is selected |
| Visualization | Three.js / React Three Fiber for web visualization, if selected |
| AI / ML | A Java-compatible framework such as Tribuo or DJL, if predictive features are implemented |

| Area | Proposed Technology |
|---|---|
| Testing | JUnit 5 + Mockito |
| Version control | Git + GitHub |
| Deployment | Docker / Docker Compose, optional |

# 10. Constraints & System Specifications

## 10.1 Technical Constraints

• The simulation core shall be implemented primarily in Java.

• The architecture should remain modular so that the visualization layer can be changed without rewriting the simulation engine.

• The system should operate on a standard modern development workstation.

• External services should not be mandatory for the core simulation.

## 10.2 Data Constraints

The initial city and its events may be generated synthetically. Real-world city data is optional and is not required for the core project.

## 10.3 Quality Constraints

The system should support graceful handling of invalid agent states, configurable random seeds, logging, and recovery from non-critical agent failures.

## 10.4 Hardware Constraints

A practical development target is a modern multi-core CPU, 16 GB RAM minimum, and a GPU capable of supporting the selected visualization framework. Actual capacity will be benchmarked rather than assumed.

# 11. Project Planning

## 11.1 Methodology

The project may be organized using Agile/Scrum principles with short development iterations and frequent demonstrations. The schedule should prioritize a working simulation core before optional advanced features.

## 11.2 Proposed Phases

| Phase | Focus | Main Deliverables |
|---|---|---|
| 1 | Analysis & Architecture | Requirements, UML diagrams, project structure, domain model |
| 2 | City & Agent Core | City model, citizens, vehicles, businesses, simulation clock |
| 3 | Algorithms & Events | A* routing, traffic model, event engine, concurrency |
| 4 | Persistence & Analytics | PostgreSQL integration, historical events, KPIs |
| 5 | What-If Engine | State snapshots, scenario branching, comparison |
| 6 | Visualization | Interactive dashboard, city map, live WebSocket updates |
| 7 | AI Extension | Prediction/recommendation module if time permits |
| 8 | Testing & Finalization | Performance tests, documentation, final demonstration |

Optional technologies such as advanced machine learning or 3D visualization should be added only after the core simulation is stable.

# 12. Testing & Validation Strategy

## 12.1 Test Typology

• Unit tests for agents, routing, economic rules, event processing, and scenario calculations.

• Integration tests for Spring Boot services and PostgreSQL persistence.

• Concurrency tests for simulation updates and shared state.

• Performance benchmarks measuring agents processed per second and memory usage.

• End-to-end tests covering a complete simulation scenario.

## 12.2 Acceptance Criteria

• The simulation can run a configured city without critical errors.

• Thousands of agents can be simulated concurrently on the development machine.

• A* routing produces valid paths through the city graph.

• What-If scenarios can be created from a previous state and compared with the baseline.

• Live simulation information is visible in the interface.

• Historical events can be inspected and replayed.

• Core modules have automated tests covering their principal behavior.

# 13. Project Deliverables

• Complete documented source code.

• Software Requirements Specification (this document).

• UML diagrams and architecture documentation.

• Database schema and initialization scripts.

• Executable application / deployment instructions.

• User and administrator documentation.

• Test report and performance benchmark results.

• Final presentation and live demonstration.

# 14. Risk Management

| Risk | Severity | Mitigation |
|---|---|---|
| Simulation becomes too complex | High | Build the core agent engine first and add optional systems incrementally. |
| Performance degradation with many agents | High | Use profiling, efficient data structures, controlled concurrency, and batched updates. |
| Scenario branching consumes too much memory | Medium | Use snapshots selectively and store only necessary state. |
| Visualization slows down with many entities | Medium | Use level-of-detail, aggregation, filtering, and selective rendering. |
| AI feature takes too long | Medium | Keep AI as an extension after the deterministic simulation and analytics are complete |
| Database becomes a bottleneck | Medium | Batch event persistence and separate live state from historical storage. |

# 15. Conclusion

AETHER is designed as a serious Java engineering project rather than a conventional CRUD application. Its central innovation is a living multi-agent city in which individual decisions create system-level behavior.

By combining concurrent Java simulation, graph algorithms, event processing, data persistence, interactive visualization, scenario branching, and optional AI analytics, the project demonstrates a broad

range of software engineering and computer science concepts while remaining achievable as a student project.

The What-If engine and historical replay provide the strongest demonstration features: users can change the city's future, compare outcomes, and understand how complex behavior emerged from the simulation.

# 16. Bibliography & References

Ferber, J. (1999). Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence. Addison-Wesley.

Batty, M. (2018). Inventing Future Cities. MIT Press.

Goetz, B., et al. (2006). Java Concurrency in Practice. Addison-Wesley Professional.

Russell, S. & Norvig, P. Artificial Intelligence: A Modern Approach.

ISO/IEC/IEEE 29148. Systems and software engineering — Requirements engineering.

# Appendix A — Glossary

### Agent

An autonomous software entity that has a state, perceives its environment, and performs actions according to defined rules.

### Multi-Agent System

A system composed of multiple interacting autonomous agents.

### Digital Twin

A digital representation of a real or modeled system whose state can evolve over time. In AETHER, the city is a simulated environment rather than a live physical city.

### What-If Simulation

A method for creating an alternative future by changing selected variables or policies from a known simulation state.

### Simulation Event

A timestamped occurrence that changes or describes the state of the simulated city.

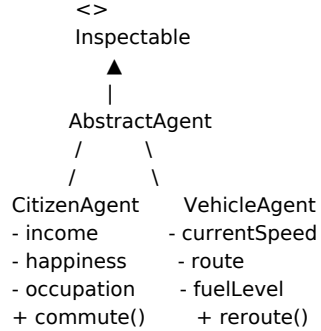
### Emergent Behavior

System-level behavior that results from interactions between individual agents rather than being directly scripted.

### Simulation State

The complete or partial set of values describing the city and its agents at a particular simulation time.

# Appendix B — Conceptual UML



BusinessAgent
- revenue
- employees
- demand
+ hire()
+ produce()

The conceptual model can be extended with City, Road, Building, Event, SimulationEngine, Scenario, Snapshot, and Analytics components during implementation.

# Project Sign-Off

Project: AETHER — Autonomous Multi-Agent Urban Simulation & Decision Intelligence Platform

Creators: Louay Hamdi & Yassine Mezghani

Version: 1.0